

4.2 利用pandas进行数据分析

4.2.1 重复，缺失与异常

在使用pandas的DataFrame处理数据时，处理数据重复、数据缺失和异常值是数据清洗和预处理的重要步骤。以下是如何分别处理这些问题的详细指导：

1. 处理数据重复

在DataFrame中，你可能会遇到重复的行，这些行可能是由于数据收集或录入过程中的错误导致的。pandas提供了 `drop_duplicates()` 方法来删除重复的行。

```
import pandas as pd

# 假设df是你的DataFrame
# 删除全部重复的行，仅保留第一次出现的行
df_no_duplicates = df.drop_duplicates()

# 如果你想根据特定的列来判断重复项，可以传入`subset`参数
df_no_duplicates_subset = df.drop_duplicates(subset=['column1', 'column2'])

# 如果你想保留重复项中的最后一次出现的行，可以设置`keep='last'`
df_last_duplicates = df.drop_duplicates(keep='last')
```

2. 处理数据缺失

数据缺失是数据清洗中常见的问题。pandas提供了多种处理缺失数据的方法，包括填充、删除或标记缺失值。

你可以使用 `fillna()` 方法用特定值或前向/后向填充来填充缺失值。

```
# 用0填充所有缺失值
df_filled = df.fillna(0)

# 用前一个有效值填充（前向填充）
df_ffill = df.fillna(method='ffill')

# 用后一个有效值填充（后向填充）
df_bfill = df.fillna(method='bfill')
```

如果缺失值太多或你认为它们对分析没有帮助，你可以选择删除含有缺失值的行或列。

```
# 删除含有任何缺失值的行
df_no_missing_rows = df.dropna()

# 删除含有任何缺失值的列
df_no_missing_cols = df.dropna(axis=1)

# 你可以设置`thresh`参数来指定保留至少有多少个非缺失值的行
df_thresh = df.dropna(thresh=2) # 例如，保留至少有两个非缺失值的行
```

3. 处理异常值

异常值（也称为离群点）是那些与其他数据点显著不同的观测值。处理异常值的方法取决于你的具体数据和你的分析目标。

通常，你可以通过描述性统计（如均值、中位数、标准差）或图形化方法（如箱线图）来识别异常值。这里有一个方法，就是通过分位点判断。计算出上四分位点Q1和下四分位点Q3后我们可以顺势得到一个IQR，异常值就是处在小于 $Q3 - 1.5IQR$ 或大于 $Q1 + 1.5IQR$ 的区段内。形如：

```
# 假设我们有一个数值列
df = pd.DataFrame({'Value': [1, 2, 3, 4, 100]})

# 计算IQR
Q1 = df['Value'].quantile(0.25)
Q3 = df['Value'].quantile(0.75)
IQR = Q3 - Q1

# 识别异常值（例如，小于Q1-1.5*IQR或大于Q3+1.5*IQR的值）
outliers = (df['Value'] < (Q1 - 1.5 * IQR)) | (df['Value'] > (Q3 + 1.5 * IQR))

# 替换异常值（例如，用中位数替换）
median_value = df['Value'].median()
df['Value'][outliers] = median_value
```

一旦识别出异常值，你可以选择删除它们、用均值、中位数或其他统计量来替换它们，或者根据具体情况进行其他处理。

```
# 删除异常值
df_clean = df[~((df['column'] > threshold) | (df['column'] < another_threshold))]

# 用均值替换异常值
df['column'] = df['column'].apply(lambda x: x if x <= threshold else mean)

# 其他处理方法...
```

在处理异常值时，要格外小心，因为异常值有时可能包含重要的信息，简单地删除或替换它们可能会丢失这些信息。因此，在决定如何处理异常值之前，最好先了解数据的背景和异常值的可能原因。

4.2.2 数据的规约

数据的规约（Normalization 或 Standardization）是数据预处理中非常重要的一步，它可以帮助改善机器学习模型的性能，尤其是那些对特征尺度敏感的算法。下面将详细介绍两种常见的规约方式：最小-最大规约（Min-Max Normalization）和Z-分数规约（Z-Score Standardization），并在pandas中实现它们。

1. 最小-最大规约（Min-Max Normalization）

最小-最大规约通过将原始数据线性变换到新的范围（通常是[0, 1]）来规约数据。其公式为：

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

其中， X 是原始数据， $X_{\min}$ 和 $X_{\max}$ 分别是原始数据中的最小值和最大值， X_{norm} 是规约后的数据。

```
import pandas as pd
import numpy as np

# 假设df是你的DataFrame, 'feature'是你想要规约的特征列
df = pd.DataFrame({
    'feature': [1, 2, 3, 4, 5]
})

# 最小-最大规约
df['feature_norm'] = (df['feature'] - df['feature'].min()) / (df['feature'].max() - df['feature'].min())

print(df)
```

2. Z-分数规约 (Z-Score Standardization)

Z-分数规约，也称为标准化，通过将原始数据减去其均值然后除以其标准差来规约数据。其公式为：

$$Z = \frac{X - \mu}{\sigma}$$

其中， X 是原始数据， μ 是原始数据的均值， σ 是原始数据的标准差， Z 是规约后的数据。

```
import pandas as pd

# 假设df是你的DataFrame, 'feature'是你想要规约的特征列
df = pd.DataFrame({
    'feature': [1, 2, 3, 4, 5]
})

# Z-分数规约
df['feature_std'] = (df['feature'] - df['feature'].mean()) / df['feature'].std()

print(df)
```

注意：如果数据的标准差为0（即所有值都相同），则除以标准差会导致除以零的错误。在实际应用中，应该处理这种情况，例如通过添加一个小的常数（如 `epsilon`）到标准差中来避免除以零。

这两种规约方式各有优缺点。最小-最大规约适用于你知道数据的范围并且希望数据被规约到特定区间的情况。而Z-分数规约不依赖于数据的具体范围，更适用于那些假设数据服从正态分布或近似正态分布的情况。选择哪种规约方式取决于你的具体需求和数据的特点。

4.2.3 数据的统计描述

当然，pandas提供了丰富的功能来查看DataFrame中的统计信息、计算统计指标，以及进行分组或透视图分析。下面我将分别介绍这些功能。

1. 查看统计信息

对于DataFrame，你可以使用 `.describe()` 方法来快速查看数值型列的统计信息，包括计数、均值、标准差、最小值、四分位数和最大值。默认情况下，`.describe()` 方法仅对数值型列进行汇总，但你可以通过 `include` 参数指定其他类型（如 `'object'` 或 `'all'`）的列。

```
import pandas as pd

# 假设df是你的DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': ['a', 'b', 'c', 'd', 'e']
})

# 查看数值型列的统计信息
print(df.describe())

# 查看所有类型的列的信息（注意：对于非数值型列，输出可能不如数值型列有用）
print(df.describe(include='all'))
```

2. 计算统计指标

除了 `.describe()` 方法外，pandas还允许你使用聚合函数（如 `mean()`，`std()`，`min()`，`max()`，`quantile()` 等）来计算DataFrame或Series的统计指标。这些函数可以直接应用于整个DataFrame、特定的列或分组后的数据。

```
# 计算列A的均值
mean_A = df['A'].mean()
print(mean_A)

# 计算整个DataFrame的均值（仅数值型列）
mean_df = df.mean()
print(mean_df)

# 计算标准差
std_df = df.std()
print(std_df)
```

3. 分组分析

pandas的 `groupby` 方法允许你根据一个或多个键对DataFrame进行分组，并对每个组应用聚合函数。这是进行复杂数据分析的强大工具。

```
# 假设我们有一个额外的列 'Category'
df['Category'] = ['X', 'Y', 'X', 'Y', 'X']

# 根据 'Category' 列分组，并计算每组的均值
grouped = df.groupby('Category').mean()
print(grouped)

# 你也可以对分组后的数据进行更复杂的操作，比如计算每个组的最大值和最小值
grouped_stats = df.groupby('Category').agg({'A': ['min', 'max'], 'B': 'mean'})
print(grouped_stats)
```

4. 透视图分析

虽然pandas本身没有直接名为“透视图”的函数（像Excel中的PivotTable那样），但你可以通过组合 `groupby`、`pivot`（对于某些特定情况）和 `pivot_table` 方法来达到类似的效果。`pivot_table` 方法特别强大，它允许你指定索引、列和值，并自动处理重复项。

```
# 使用pivot_table创建一个简单的透视图
pivot_table = df.pivot_table(values='A', index='Category', aggfunc='mean')
print(pivot_table)

# 你也可以指定多个索引和列，以及不同的聚合函数
pivot_table_complex = df.pivot_table(values='A', index='Category', columns=df.index, aggfunc='mean')
# 注意：上面的代码可能不会按预期工作，因为columns=df.index通常不是有效的用法。
# 你应该指定一个具体的列或列列表作为列参数。

# 一个更实际的例子，假设我们有一个额外的'Year'列
df['Year'] = [2020, 2020, 2021, 2021, 2022]
pivot_table_year = df.pivot_table(values='A', index='Category', columns='Year', aggfunc='mean')
print(pivot_table_year)
```

通过这些方法，你可以对DataFrame进行深入的统计分析和数据透视。